Лабораторна робота № 9

Тема. Модулі та пакети. Створення віртуальних оточень, керування залежностями за допомогою pip та створення власного wheel-пакету.

Мета: набути практичних навичок роботи з модулями та пакетами, створення віртуальних середовищ, встановлення та керування зовнішніми залежностями, а також створення власного wheel-пакету.

Короткі теоретичні відомості

Модуль – це файл із кодом Python, який містить визначення функцій, класів, змінних та виконуваний код. Ім'я модуля – це ім'я файлу без розширення .py.

Модулі дозволяють: повторно використовувати код; організувати код; уникати конфліктів імен.

Пакет – це каталог, який містить спеціальний файл __init__.py та один або кілька модулів (або підпакетів). Пакети дозволяють організувати пов'язані модулі в ієрархічну структуру. Файл __init__.py виконується при імпорті пакета та може містити код ініціалізації або визначати, що експортується.

Способи імпорту: *import module* – імпорт всього модуля; *from module import name* – імпорт конкретних елементів; *import module as alias* – імпорт з псевдонімом; *from module import* * – імпорт всього (не рекомендується); для імпорту модулів з пакетів використовується крапкова нотація, наприклад *import package.module*. Python шукає модулі в такому порядку:

1. поточна директорія;
2. директорії, вказані в змінній середовища PYTHONPATH;
3. директорії стандартної бібліотеки;
4. директорії встановлених сторонніх пакетів.

Кожен модуль має атрибут *__name__*: якщо модуль запущено безпосередньо *__name__* дорівнює *"__main__"*; якщо модуль імпортовано – *__name__* дорівнює імені модуля. Це дозволяє писати код, який виконується тільки при прямому запуску:

```
def main():
    print("Код виконується")
```

```
if __name__ == "__main__":
    main()
```

Абсолютний імпорт – використовує повний шлях від кореня пакета *from graphics.shapes.circle import area*.

Відносний імпорт – використовує крапки для позначення поточного та батьківського рівнів. Приклад:

```
# У модулі graphics/shapes/circle.py
from . import rectangle # імпорт сусіднього модуля
from .. import geometry # імпорт з батьківського пакета
```

Віртуальне середовище (virtual environment) – це ізольоване середовище Python, яке містить власну копію інтерпретатора та незалежний набір встановлених пакетів. venv – інструмент (вбудований модуль) для створення віртуальних середовищ. Приклад використання у терміналі (командному рядку):

```
# Створення віртуального середовища
python -m venv .venv

# Активація (Windows)
.venv\Scripts\activate

# Активація (Linux)
source .venv/bin/activate

# Деактивація
deactivate
```

pip – це менеджер пакетів, який використовується для встановлення, оновлення та видалення бібліотек (пакетів). Базові команди pip:

```
# Встановлення пакета
pip install package_name

# Встановлення конкретної версії
pip install package_name=1.2.3

# Встановлення з файлу залежностей
pip install -r requirements.txt
```

```
# Оновлення пакета
pip install --upgrade package_name

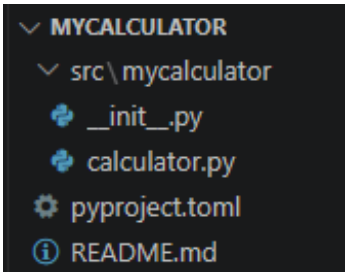
# Видалення пакета
pip uninstall package_name

# Перегляд встановлених пакетів
pip list

# Запис всіх встановлених у поточному середовищі пакетів з їхніми версіями
pip freeze > requirements.txt
```

PyPI (Python Package Index) – це офіційний публічний репозиторій пакетів Python. Це централізоване сховище, де розробники можуть публікувати свої пакети, а користувачі – встановлювати їх за допомогою pip.

Wheel-пакет (.whl) – це готовий для встановлення пакет Python у бінарному форматі. Щоб створити власний wheel-пакет, потрібно підготувати структуру проєкту, додати конфігурацію пакета і зібрати його за допомогою інструмента build.



Приклад структури проєкту

pyproject.toml – це основний файл опису пакета. Приклад:

```
[build-system]
requires = ["setuptools>=61", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "mycalculator"
version = "1.0.0"
```

```
description = "Simple calculator package"
readme = "README.md"
requires-python = ">=3.9"
authors = [
    {name = "Your name"}
]

[tool.setuptools.packages.find]
where = ["src"]
```

Для створення wheel-пакета потрібно у віртуальному середовищі встановити інструмент build, перейти до папки проєкту та виконати команду *python -m build*. Після успішної збірки у папці dist з'явиться файл із розширенням .whl.

Коли wheel-пакет створено його можна встановити через pip, вказавши шлях до файлу .whl.

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.
2. Об'єднати код із попередніх виконаних лабораторних робіт у вигляді модулів та пакетів, створити структуру Python-проєкту та додати файл опису проєкту для подальшої збірки wheel-пакета. Після створення wheel-пакета виконати його встановлення в нове віртуальне середовище, використати встановлений пакет в модулі та запустити його на виконання.
3. Написати звіт з виконання лабораторної роботи.

Контрольні питання

1. Чим відрізняється модуль від пакета?
2. Для чого використовується файл requirements.txt?
3. Для чого потрібен wheel-пакет?
4. Для чого потрібен файл __init__.py у пакеті?
5. Яка команда створює віртуальне середовище?